Deep Hydro-Sim

How to attain robust performance in complex and dynamic environment is one of the core research topics in robotics nowadays. It is also main challenge AUV(Automatic Unmanned Vehicle) face in order to perform underwater tasks and motion control. Reinforcement Learning is demonstrating its magical power in robotics training, however, main stream training platforms like Mujoco and Gym focuses on rigid body physic simulation, lacking the fluid dynamic consideration which is crucial in AUV simulation and training, leading to a dilemma in Sim2Real implementation. While the high precision underwater simulator like Stonefish and UUV simulator fails to support large scale RL training. In this experiment, we aim to take advantage of both and construct a reliable platform and AUV targeted RL method. Taking the collected data in stonefish as ground truth and compute the nonlinear fluid dynamic part by subtracting the known part such as gravity and buoyancy. Then input the data into a MLP deep neural network to train a model that predicts the component that are nonlinear and difficult to explicitly compute. Eventually combine the model with Mujoco physic simulator and apply main reinforcement algorithm like PPO to test the agent and the training platform.

▼ Physical Framework

6-DOF rigid body system

$$\dot{\boldsymbol{\nu}} = [\ \underbrace{\dot{u}, \dot{v}, \dot{w}}_{\text{线加速度 (m/s}^2)}\ ,\ \underbrace{\dot{p}, \dot{q}, \dot{r}}_{\text{角加速度 (rad/s}^2)}\ ]^T$$

$$\boldsymbol{\nu} = [\ \underbrace{u, v, w}_{\text{线速度 (m/s)}}\ ,\ \underbrace{p, q, r}_{\text{角速度 (rad/s)}}\ ]^T$$

$$\boldsymbol{\tau} = [F_x, F_y, F_z, \tau_\phi, \tau_\theta, \tau_\psi]^T$$

Effective mass = Robot mass + added mass (Inertial of the water pushed)

$$\mathbf{M}_{rb} = \begin{bmatrix} m & 0 & 0 & 0 & z_g m & -y_g m \\ 0 & m & 0 & -z_g m & 0 & x_g m \\ 0 & 0 & m & y_g m & -x_g m & 0 \\ 0 & -z_g m & y_g m & I_{xx} & -I_{xy} & -I_{xz} \\ z_g m & 0 & -x_g m & -I_{yx} & I_{yy} & -I_{yz} \\ -y_g m & x_g m & 0 & -I_{zx} & -I_{zy} & I_{zz} \end{bmatrix}$$

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{g}(\boldsymbol{\eta}) = \boldsymbol{\tau}_{\text{prop}}$$

Kinetic equations (equilibrium state):

- $\mathbf{M}$: System inertia matrix (Rigid body + Added mass).
- $\mathbf{C}(\boldsymbol{\nu})$: Coriolis and centripetal matrix.
- $\mathbf{D}(\boldsymbol{\nu})$: Hydrodynamic damping matrix (Linear + Quadratic).
- $\mathbf{g}(\boldsymbol{\eta})$: Vector of gravitational and buoyancy forces.
- $\boldsymbol{\tau}_{\text{prop}}$: Propulsion forces (Thrusters).

物理项	符号	归属方	实现方式	理由
刚体惯性	$M_{rb}\dot{\nu}$	**MuJoCo**	XML `<inertial>`	刚体动力学是 MuJoCo 的老本行，极准。
附加质量	$M_A\dot{\nu}$	**MuJoCo**	修改 XML 质量	将"虚拟水质量"直接叠加在刚体质量上，保证积分器稳定。
重力	W	**MuJoCo**	XML `gravity`	基础物理，自带。
浮力	B	**Python公式**	代码计算 → `qfrc`	MuJoCo 没有流体静力学，需要你手写"反重力"公式注入。
刚体科氏力	$C_{rb}\nu$	**MuJoCo**	原生自带	基于牛顿-欧拉方程，只要有质量和旋转，它自动会算。
流体科氏力	$C_A\nu$	**神经网络**	NN 预测 → `qfrc`	复杂的流体惯性耦合，公式难写，交给 AI 学。
流体阻力	$D(\nu)\nu$	**神经网络**	NN 预测 → `qfrc`	高度非线性（粘滞+涡流），是 AI 学习的核心目标。
碰撞/接触	Contact	**MuJoCo**	原生自带	触底、撞墙、机械臂抓取，MuJoCo 强项。
推力分配	$\tau \to u$	**Python公式**	伪逆矩阵 $\mathbf{B}^\dagger$	属于控制算法层（Control Allocation），不属于物理层。

This physic shows what data and status value should be collected in stonefish and what the inputs and outputs should be in the MLP deep

neural network.

Additionally, the mass matrix is also derived in stonefish output.

- ▼ Stonefish
  - ▼ Sampling strategy：

数据类型	描述	建议时长	数据量 (100Hz)	目的
单轴扫频	前后、左右、上下、自旋单独运动 (Chirp)	15 分钟	90,000	学习主对角线阻力 ($F_x \propto u^2$)
耦合机动	螺旋上升、侧滑、斜向运动	15 分钟	90,000	学习科里奥利力和耦合项 ($F_y \propto u \cdot r$)
随机暴走	所有推进器给随机信号 (Random Walk)	30 分钟	180,000	填充状态空间的空白区域，防止过拟合
总计		**60 分钟**	**360,000 条**	**完美数据集**

Cover the state space as comprehensively as possible

1. **平滑与微分 (Preprocessing)**：
   - 先对速度 $\boldsymbol{\nu}$ 进行 Savitzky-Golay 滤波。
   - 再算出加速度 $\dot{\boldsymbol{\nu}}$。
   - 再反算 Label 力。
2. **打乱 (Shuffle)**：
   - **非常重要！** 原始数据是时间序列（连续的），直接训练会导致网络在每个 Batch 里只学到某种特定运动。
   - 必须在 PyTorch `DataLoader` 里设置 `shuffle=True`，彻底打乱时间顺序。
3. **划分数据集 (Split)**：
   - **Training Set (80%)**: 28.8万条。
   - **Validation Set (20%)**: 7.2万条。**一定要看 Validation Loss**，只有它下降了，才说明你的模型学会了物理规律，而不是死记硬背。
4. **归一化 (Normalization)**：
   - 速度范围是 [-2, 2]，力可能是 [-100, 100]。
   - 必须使用 `StandardScaler` (减均值，除方差) 将输入输出都缩放到 0 附近。保存好 `mean` 和 `std`，将来在 MuJoCo 里推理时要用同样的参数反归一化。

Stochastic Sampling strategy：

方案 1：APRBS (推荐 - 最简单有效)

逻辑：随机选一个目标推力，保持一段时间（比如 2~5 秒），然后再随机切到下一个目标推力。

- **特点**：像阶跃信号一样。能让机器人迅速加速到极值，覆盖高速区间。
- **C++ 实现逻辑**：

方案 2：平滑随机游走 (Smoothed Random Walk / Ornstein-Uhlenbeck)

逻辑：模拟人类驾驶员推操纵杆。不是瞬间跳变，而是连续平滑地随机游荡。

- **特点**：数据分布更均匀，不会像 APRBS 那样只有急停急起。
- **实现**：使用一阶低通滤波器过滤白噪声。

方案 3：组合正弦波 (Sum of Sines) - 专门针对耦合

逻辑：让不同的推进器以不同的频率正弦波动。

- 比如：推力 $X = \sin(0.1t)$，推力 $Y = \sin(0.3t)$。
- **特点**：这样能保证 X 和 Y 既有同时为大的时候，也有一个大一个小的时候，彻底遍历 (u, v) 的组合空间。

▼ Model import

▼ Test Demo

UnderwaterTest:

1. **sf::BodyPhysicsMode::SUBMERGED**: 只有这个模式才包含**水下物理**（浮力、水阻力、附加质量）。这是 AUV 仿真的灵魂。
2. **sf::Thruster**: 最核心的组件。它模拟了螺旋桨在水中的推力特性（电压 -> 转速 → 推力），而不是简单的给一个力。
3. **sf::FeatherstoneRobot**: 最适合浮动基座机器人（AUV/无人机）的动力学解算器。
4. **sf::Compound**: 允许你把船体(Hull)、电池舱、支架拼装成一个整体刚体。
5. **传感器集成**: 代码里演示了 IMU, DVL, Camera, Sonar 的标准写法。
6. **SimulationStepCompleted**: 这里是写控制算法（PID, 推力分配）的唯一入口。

FloatingTest:

1. **hull→ScalePhysicalPropertiesToArbitraryMass(150.0)**:
   - *痛点*：在 UnderwaterTest 里，你需要设定材料密度，然后物理引擎根据体积算质量。这很麻烦，因为你通常只知道你的机器人总重（比如 20kg），而不知道那个 CAD 模型的体积是多少。
   - *用法*：直接把这行代码拷过去，填上你机器人的真实重量（例如 20.0），Stonefish 会自动反向调整密度。
2. 推进器运动模式设置

JointsTest：

1. **sf::RevoluteJoint + sf::Servo**:
   - 如果你的推进器是可以旋转的（矢量喷口）。

    - 或者你的 AUV 底部带有一个**机械臂**。
  2. **sf::SuctionCup**:
    - 如果你需要做抓取任务，或者吸附在船体/管壁上。
  3. **简单的几何体 (sf::Box, sf::Sphere)**:
    - 用来快速生成海底的障碍物、目标物，而不需要每次都去画 CAD 模型。

  SlidingTest：

  1. **sf::Odometry + setRenderable(true)**:
    - *功能*：它能在 3D 窗口里画出一条红色的**轨迹线**。
    - *用法*：把这个传感器挂在你的 AUV 上，你就能直观地看到你的路径规划算法走得直不直，有没有漂移。

▼ Mujoco